

Práctica 4 - Capa de Transporte

Revisión 2.4

1. ¿Cuáles son las funciones de la capa de transporte en el modelo OSI? ¿Cuáles implementan TCP y cuáles UDP? ¿Existen otros protocolos de transporte en el stack TCP/IP? ¿Cuáles son los más utilizados? ¿En qué protocolo se encapsulan y en qué parte de un sistema se los suele encontrar: en el kernel del SO, en espacio de usuario, cómo microservicios fuera del kernel?
2. Indique que asumen TCP y UDP con respecto a los servicios provistos por la capa de Red/Internetworking sobre la cual se implementan. ¿Cuál es el campo del datagrama IP y los valores utilizados para diferenciarlos en la multiplexación (Hint: buscar en `/etc/protocols`)?
3. La PDU de la capa de transporte es nombrada de forma genérica segmento. Indique para los protocolos indicados en los puntos anteriores cómo se llaman específicamente las unidades de datos y realice un diagrama de su estructura.
4. Responder:
 - a) ¿Qué sucede si llega un datagrama IPv6 a un host y éste no tiene implementado IPv6?
 - b) ¿Qué sucede si llega un segmento TCP a un host que no tiene soporte de este protocolo de transporte?
 - c) ¿Qué sucede si llega un segmento TCP a un host y éste no tiene un proceso esperando en el puerto destino indicado? ¿Qué sucede si el mensaje es UDP?
5. ¿En qué se diferencian los checksum de UDP, TCP, IPv4 e IPv6?
6. Un proceso desde 10.0.0.10 inicia una conexión TCP, por ejemplo mediante el comando telnet, hacia un servidor 192.168.1.10. Al mismo tiempo otro proceso desde 10.0.0.10 inicia otra conexión TCP hacia el mismo servicio. Ambas conexiones se establecen.
 - a) Indicar direcciones y números de puerto origen y destino de la primera conexión.
 - b) Indicar direcciones y números de puerto origen y destino para la segunda conexión.
 - c) Si genera una nueva conexión desde 10.0.0.11 hacia el mismo servicio, indicar direcciones y números de puerto origen y destino utilizados.
 - d) Indicar la cantidad de segmentos TCP transmitidos en total.
 - e) Indicar los "flags" que se verán en los segmentos TCP transmitidos.
 - f) Indicar cantidad de segmentos TCP totales (inicio, datos y cierre) en caso que solo dos conexiones transmiten 1 byte c/u y luego cierran las 3.
 - g) Indicar la cantidad de datagramas transmitidos en caso de que las transferencias se realicen utilizando UDP.
 - h) Hacer una simulación de los escenarios, con nc (netcat) o telnet.

7. Dada la siguiente salida del comando (netstat o ss), responder:

```
# netstat -atun
Active Internet connections (servers and established)
Proto R-Q S-Q Local Address      Foreign Address    State
tcp    0  0  127.0.0.1:43695    0.0.0.0:*          LISTEN
tcp    0  0  127.0.0.1:7634    0.0.0.0:*          LISTEN
tcp    0  0  0.0.0.0:22        0.0.0.0:*          LISTEN
tcp    0  0  127.0.0.1:631     0.0.0.0:*          LISTEN
tcp    0  0  0.0.0.0:17500     0.0.0.0:*          LISTEN
tcp    0  0  0.0.0.0:8000      0.0.0.0:*          LISTEN
tcp    0  0  127.0.0.1:40963   0.0.0.0:*          LISTEN
tcp    0  0  127.0.0.1:2628    0.0.0.0:*          LISTEN
tcp    0  0  10.168.1.163:51222 173.194.42.54:443  ESTABLISHED
tcp    0  0  127.0.0.1:43695   127.0.0.1:45547    ESTABLISHED
tcp    1  0  10.168.1.163:50120 191.189.39.14:80   CLOSE_WAIT
tcp    0  0  127.0.0.1:8000    127.0.0.1:35250    ESTABLISHED
tcp    0  0  127.0.0.1:35250   127.0.0.1:8000     ESTABLISHED
tcp    0  1  10.168.1.163:45123 1.1.1.1:9000       SYN_SENT
tcp    0  0  10.168.1.163:40123 99.59.148.17:443   TIME_WAIT
tcp    1  1  10.168.1.163:58432 104.154.94.81:443  LAST_ACK
tcp    0  0  10.168.1.163:36121 138.160.162.116:7  ESTABLISHED
tcp    1  0  10.168.1.163:58433 205.154.94.81:443  CLOSE_WAIT
tcp    0  0  10.168.1.163:60123 91.189.89.76:443   ESTABLISHED
tcp    0  0  10.168.1.163:42133 173.194.42.33:443  ESTABLISHED
tcp    0  0  10.168.1.163:50121 199.16.156.83:443  ESTABLISHED
tcp    0  0  127.0.0.1:45547   127.0.0.1:43695    ESTABLISHED
tcp6   0  0  :::80             :::*               LISTEN
tcp6   0  0  :::22             :::*               LISTEN
tcp6   0  0  :::1:631          :::*               LISTEN
udp    0  0  0.0.0.0:68        0.0.0.0:*          *
udp    0  0  0.0.0.0:69        0.0.0.0:*          *
udp    0  0  0.0.0.0:59744     0.0.0.0:*          *
udp    0  0  0.0.0.0:17500     0.0.0.0:*          *
udp    0  0  0.0.0.0:5353      0.0.0.0:*          *
udp6   0  0  :::59695          :::*               *
udp6   0  0  :::5353           :::*               *
```

- ¿Cuántas conexiones TCP hay establecidas?
- ¿Cuántas conexiones TCP hay establecidas solo como cliente y cuántas como servidor?
- ¿Cuántas conexiones TCP están aún pendientes por establecerse?
- ¿A qué servicios bien conocidos (puertos 0 a 1023) tiene el host conexiones TCP establecidas?
- ¿Qué servicios bien conocidos (TCP y UDP) tiene corriendo el host (esperando por recibir información)? (Ver RFC1700)
- ¿Cuántas conexiones están en proceso de cierre?
- ¿Cuáles conexiones TCP tuvieron el cierre iniciado por el host local y cuáles por el remoto?
- ¿En qué puertos podría recibir datos sobre IPv6 desde otros hosts, tanto TCP6 como UDP6?

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.1.1	172.20.1.100	TCP	74	41749 > vce [] Seq= Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=270132 TSecr=0
2	0.001264	172.20.1.100	172.20.1.1	TCP	74	vce > 41749 [SYN, ACK] Seq=1047471501 Ack=3933822138 Win=5792 Len=0 MSS=1460 SACK_PERM=1
3	0.001341			TCP	66	> [] Seq= Ack= Win=5888 Len=0 TSval=270132 TSecr=1877442

▶ Internet Protocol Version 4, Src: 172.20.1.100 (172.20.1.100), Dst: 172.20.1.1 (172.20.1.1)
 ▼ Transmission Control Protocol, Src Port: vce (11111), Dst Port: 41749 (41749), Seq: 1047471501, Ack: 3933822138, Len: 0
 Source port: vce (11111)
 Destination port: 41749 (41749)
 [Stream index: 0]
 Sequence number: 1047471501
 Acknowledgement number: 3933822138
 Header length: 40 bytes
 ▼ Flags: 0x012 (SYN, ACK)
 000. = Reserved: Not set
 ...0 = Nonce: Not set
 0... = Congestion Window Reduced (CWR): Not set
0.. = ECN-Echo: Not set
0. = Urgent: Not set
1 = Acknowledgement: Set
 0... = Push: Not set
0.. = Reset: Not set
 ▶1. = Syn: Set
 0 = Fin: Not set
 Window size value: 5792
 [Calculated window size: 5792]
 ▶ Checksum: 0x9803 [validation disabled]

Figura 1: Captura TCP 1

8. De acuerdo a la captura de la figura 1 indicar los valores de los campos que están difuminados ("blur").
9. De acuerdo a la siguiente salida del comando netstat responder:

```
# netstat -atunp
Active Internet connections (servers and established)
Proto . Local Address      Foreign Address    State
tcp    0.0.0.0:22          0.0.0.0:*          LISTEN  999/sshd
tcp    127.0.0.1:631       0.0.0.0:*          LISTEN  11351/cupsd
tcp    13.10.0.14:5217    91.189.95.73:443   CLOSE_  10418/remot
tcp    13.10.0.14:22      11.191.89.18:49357 ESTABL  11696/sshd:
tcp6   :::80              :::*               LISTEN  1847/apache2
tcp6   :::22              :::*               LISTEN  999/sshd
tcp6   :::1:631           :::*               LISTEN  11351/cupsd
```

- a) ¿Cuál es el proceso asignado al port de servicios de impresión de red? (Hint: buscar en /etc/services).
- b) ¿Se podría enviar a imprimir al servidor de impresión de este equipo a través de la red desde otros equipos?
- c) ¿Qué servicios pueden recibir conexiones sobre IPv6?

10. Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:

```

srv# netstat -atun | grep 4500
tcp    0      0  0.0.0.0:4500          0.0.0.0:*            LISTEN
tcp    0      0  157.0.0.1:4500      157.0.11.1:52843     SYN_RECV

cli# netstat -atun | grep 4500
tcp    0      1  157.0.11.1:52843    157.0.0.1:4500       SYN_SENT

```

- ¿Qué paquetes llegaron y cuáles parecen que se están perdiendo en la red?
- ¿Cómo quedará el estado en ambos lados si no logra establecerse la conexión?

11. Dado el diagrama de intercambio de segmentos de la figura 2 y suponiendo que se aplica el mecanismo de ARQ Go-Back-N numerando por bytes, responder:

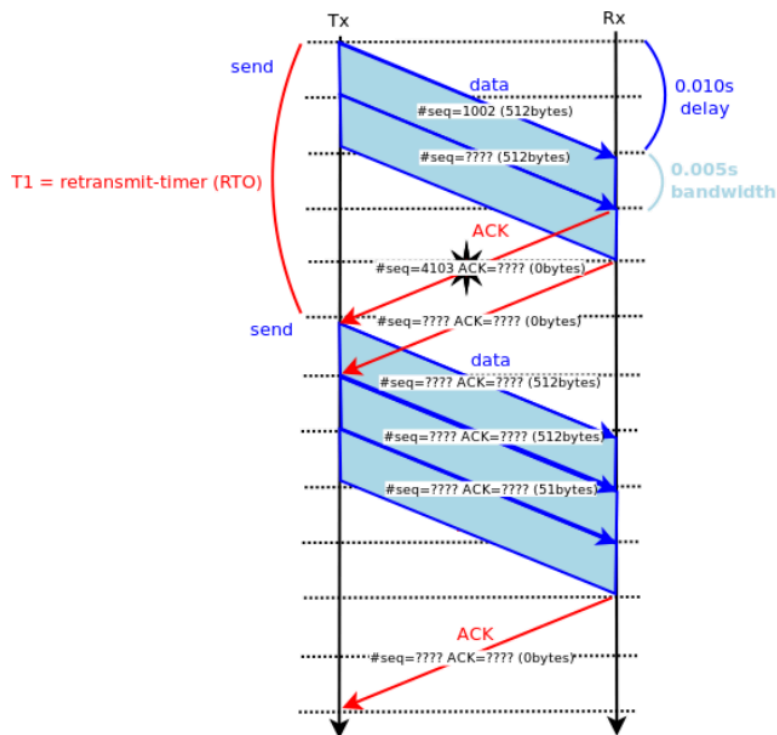


Figura 2: Control de Errores Go-Back-N

- ¿Cuál es el ancho de banda digital (throughput) que se está obteniendo?
- ¿Cuál debería ser el valor óptimo del T1 (RTO, Timer de retransmisión) suponiendo que el delay se mantiene constante? ¿Cómo se soluciona cuando el delay no es constante?

- c) ¿Cuál sería un tamaño de ventana óptimo?
- d) ¿Cuántos bytes lograron transferirse de forma efectiva?
- e) Suponiendo que el primer ACK se pierde, completar los valores indicados con signos de interrogación.
12. El host A desea establecer una sesión TCP con el host B. A selecciona un ISN de 50430, MSS de 1400 bytes y un tamaño de ventana de 64KB; B, por su parte, tiene un ISN de 68900, MSS de 1000 bytes y un tamaño de ventana de 32KB. Responder:
- a) ¿Cómo sería el intercambio de mensajes para establecer la sesión?
- b) ¿Cuántos segmentos de tamaño máximo le puede enviar A a B sin esperar un ACK? ¿Y B a A? (Considere primero que no se aplica control de congestión tradicional y luego aplicándolo).
13. Se tiene una conexión TCP entre dos hosts, A y B. B ya recibió correctamente de A todos los bytes hasta el byte 225. A se conecta desde el port 1986 al port 22 de B. Responder:
- a) ¿Qué valor indicará B en el campo ACK para reconocer esta condición? ¿Qué ports utilizará?
- b) Si A le envía dos segmentos a B de 100 y 120 bytes respectivamente, ¿cuáles son los números de secuencia de los dos segmentos?
- c) Si B envía un ACK por cada segmento recibido, ¿cuáles serían los números de ACKs seteados? ¿Y si envía uno solo por los dos segmentos?
- d) Si el segundo segmento de datos (el de 120 bytes) arriba antes que el primer segmento, en el ACK del primer segmento recibido, ¿cuál es el valor del número de ACK?
14. Dada la sesión TCP de la figura 3 completar los valores marcados con un signo de interrogación.

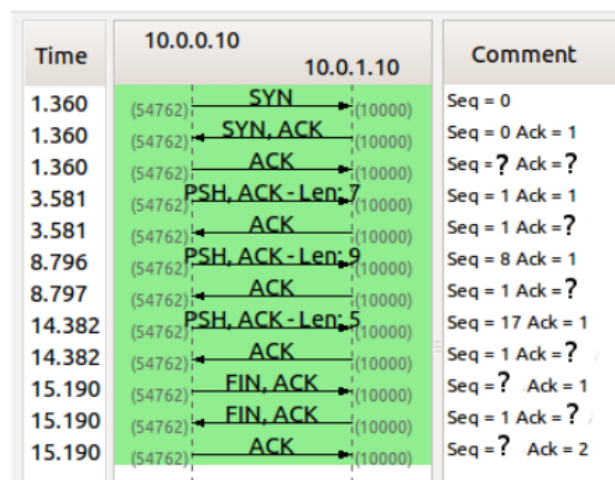


Figura 3: Sesión TCP

15. A partir de las capturas `tcp-init.pcap` y `tcp-init2.pcap` indicar qué opciones se intercambiaron en TCP. Indicar los ISN (Initial Sequence Number) de cada extremo y el port del cliente y del servidor. Investigar en la captura `tcp-init3.pcap` el intercambio del MSS (Maximum Segment Size) y comparar la diferencia con las anteriores, ¿a qué puede deberse?
16. Se tiene un enlace no congestionado con un RTT de 10ms. La ventana de recepción es de 12KB, y el $cwnd = IW = 1 \times MSS$, MSS es de 1KB. Si el Ssthresh inicia de 16KB y el emisor envía datos continuamente, ¿cuánto tiempo le toma a la ventana utilizada, $\min(cwnd, rwnd)$, alcanzar su máximo? Hacer el diagrama.
17. Suponga que la ventana de congestión de TCP está seteada en 18KB y que, en ese momento, se vence el RTO de una transmisión. Si el tamaño máximo del segmento es 1KB, ¿qué tan grande será la ventana si las próximas 4 ráfagas de transmisión son exitosas?
18. Observar el gráfico de la fig. 4, que ejecuta TCP Reno y responder:

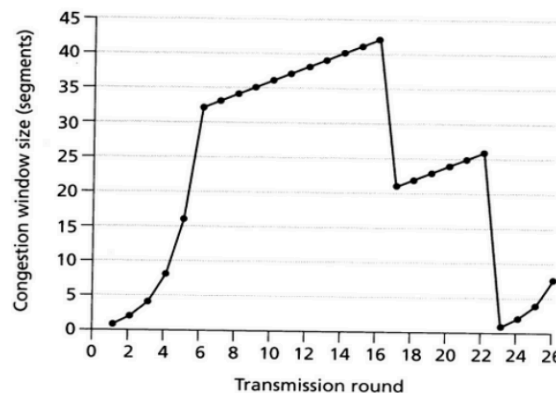


Figura 4: TCP Reno - Control de Congestión

- a) ¿Cuáles son los intervalos en los que se ejecuta Slow-Start?
 - b) ¿Cuáles son los intervalos en los que se ejecuta Congestion Avoidance?
 - c) ¿Qué evento se produce en el momento 16? ¿Y en el 22?
 - d) ¿Cuál es el valor de la variable (Ssthresh) SStreshold inicialmente? ¿Y en los momentos 18 y 24?
19. A partir de las capturas **tcp*.pcap.gz** indicar en cuáles y cuándo se detecta que se activó el control de flujo y en cuáles el control de congestión.
 20. NAT/NAPT: Dado el diagrama de la figura 5 indicar como quedaría la tabla de NAT/NAPT del router n1 si este hace "overloading" con su única IP pública y se generan las dos conexiones TCP desde los hosts n2 y n3 hacia el servidor s4.

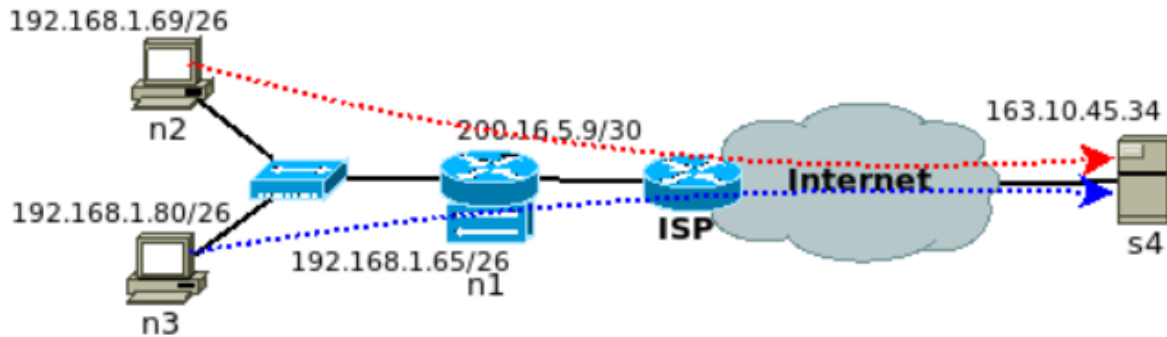


Figura 5: Diagrama de NAT

21. Para el ejercicio 7 indicar si hay indicios que se está realizando NAT/NAPT. Cómo quedaría la tabla de NAT/NATP del router por donde pasa el tráfico para las tres primeras conexiones de salida en estado ESTABLISHED en caso que se haga overloading/masquerading y en el caso que solo se haga NAT básico. (Considere que IPs debería administrar el router).

22. Ejercicios UDP a entregar. Correr sobre la topología de la figura 6:

- Levantar un servicio UDP con la herramienta nc (netcat) en el host n9 y enviar información desde n13 usando el mismo comando. Ver el estado de los sockets en ambos extremos. ¿Qué significa el estado ESTABLISHED en UDP?
- Levantar en el super daemon inetd o similar con el servicio UDP echo y probar con un cliente programado en el lenguaje de su elección mediante la API socket contra este servicio. Inspeccionar el estado. Ver de generar datos hacia el “servidor” desde más de un Nodo.
- Enviar información desde n13 a n9 a un port UDP donde no existe un proceso esperando por recibir datos. ¿Cómo notifica el stack TCP/IP de este hecho? Investigue la herramienta traceroute que ports utiliza y cómo usa estos mensajes (Ver ejercicio de IP con ruteo estático).
- Para las pruebas anteriores capturar tráfico y ver el formato de los datagramas UDP y como se encapsulan en IP.

23. Ejercicios TCP a entregar. Correr sobre la topología de la figura 6 (continuación de lo que venían realizando en la topología en la práctica anterior):

- Programar en el lenguaje de su elección mediante la API socket un servidor TCP que escuche conexiones en el puerto 9000 y que los datos que reciba los descarte. Correr en el nodo n9 y enviar datos desde otro nodo usando la herramienta nc (netcat) o telnet.
- En el nodo n9 levantar con el super daemon inetd algunos servicios extras, como tcp echo, discard y otros. Chequear los servicios TCP y UDP activos.
- Desde el nodo n13 realizar una conexión TCP y enviar datos mediante un programa cliente de su elección al servicio discard, y al echo. Capturar el tráfico con la herramienta tcpdump o wireshark y analizar la cantidad de segmentos, los flags utilizados y las opciones extras que llevan los encabezados tcp.
- Sin cerrar las conexiones chequear los servicios activos y ver los Estados.

- e) Generar nuevas conexiones hacia el nodo n9 e inspeccionar los estados. Por ejemplo realizar varias conexiones simultáneas al servicio tcp echo desde el mismo origen y desde otros nodos.
- f) Intentar generar conexiones a un puerto donde no existe un proceso esperando por recibir datos. ¿Cómo notifica TCP de este hecho (ver flags)?
- g) Cerrar las conexiones y ver el estado de los servicios en ambos lados. ¿En qué estado queda el que hace el cierre activo?
- h) Observando la captura indicar la cantidad de segmentos y los flags utilizados. ¿Con cuántos segmentos se cerró la conexión? ¿Existen otras variantes de cierre?
- i) Hacer un diagrama de los segmentos intercambiados con los números de secuencia absolutos para una de las sesiones TCP (Se puede usar la herramienta wireshark u otra).
- j) Alternativo: Realizar una conexión mediante nc indicando un puerto específico para el cliente. Luego cerrar la conexión desde el cliente e intentar abrirla nuevamente. ¿En qué estado está el socket? Investigar valor del 2MSL en la plataforma sobre la cual está haciendo los tests.

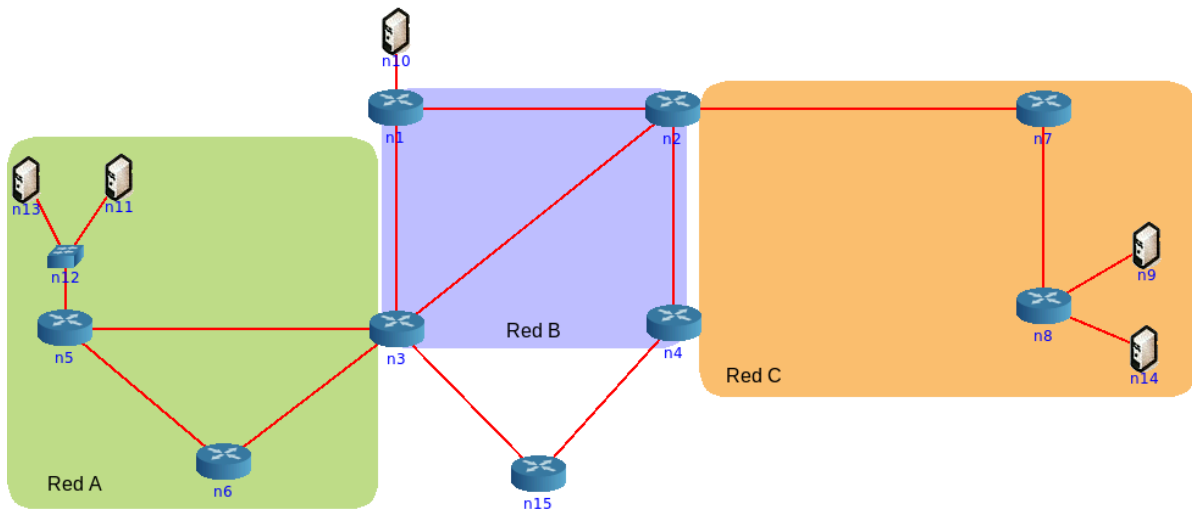


Figura 6: Diagrama de topología a entregar

Resolución y Análisis de la Práctica 4:

Capa de Transporte

Este informe presenta una resolución exhaustiva y un análisis técnico de las 23 preguntas de la Práctica 4 de Redes II, dedicada a la Capa de Transporte. El análisis abarca los fundamentos teóricos de los protocolos TCP y UDP, la gestión de conexiones, el diagnóstico de estado de red, el análisis de trazas a nivel de paquete, el control de rendimiento y congestión, y los escenarios de traducción de direcciones de red (NAT).

Sección 1: Fundamentos de la Capa de Transporte

Esta sección aborda las preguntas 1, 2, 3 y 5 de la práctica ¹, estableciendo la base teórica necesaria para el análisis de protocolos, estados y rendimiento en secciones posteriores.

1.1. Funciones y Protocolos (Pregunta 1)

Funciones de la Capa de Transporte (OSI):

La Capa de Transporte (Capa 4) del modelo OSI actúa como un puente entre las capas de aplicación (orientadas al usuario) y las capas inferiores (orientadas a la red). Sus funciones principales son 2:

1. **Segmentación y Reensamblaje:** Dividir los flujos de datos de la capa de aplicación en segmentos más pequeños para su transmisión y reensamblarlos en el destino.
2. **Direccionamiento de Procesos (Multiplexación):** Utilizar números de puerto (ej. puerto 80 para HTTP) para identificar las aplicaciones de origen y destino, permitiendo que múltiples aplicaciones en un host compartan la red simultáneamente.
3. **Control de Conexión:** Proporcionar servicios orientados a la conexión (como TCP) o no orientados a la conexión (como UDP).
4. **Control de Flujo:** Gestionar la tasa de transmisión de datos para evitar que un emisor rápido sature a un receptor lento, típicamente mediante mecanismos de "ventana deslizante".
5. **Control de Errores (Fiabilidad):** Asegurar la entrega íntegra y ordenada de los datos,

utilizando checksums para detectar corrupción y acuses de recibo (ACKs) y retransmisiones para corregir pérdidas.

Implementación en TCP vs. UDP:

- **TCP (Transmission Control Protocol):** Implementa un conjunto *completo* de estas funciones. Es un protocolo orientado a la conexión, lo que requiere un "saludo de tres vías" (3-way handshake) antes de transmitir datos.⁴ Proporciona un servicio de flujo de bytes *fiable, ordenado* y con *comprobación de errores*.⁴ Implementa mecanismos explícitos de control de flujo (mediante el campo "Window Size" de su encabezado) y control de congestión (mediante algoritmos como Slow Start y Congestion Avoidance).⁴
- **UDP (User Datagram Protocol):** Implementa el conjunto *mínimo* de funciones: direccionamiento de procesos (puertos) y comprobación de errores (checksum).⁵ Es un protocolo *no orientado a la conexión* que proporciona un servicio de datagramas "best-effort" (mejor esfuerzo): no es fiable, no garantiza el orden y no tiene control de flujo ni de congestión.⁴ Su ventaja radica en su simplicidad y bajo *overhead*.

Otros Protocolos del Stack TCP/IP:

Sí, existen otros protocolos además de TCP y UDP. Los más relevantes en la actualidad son:

- **SCTP (Stream Control Transmission Protocol):** Es un protocolo de transporte avanzado que combina las mejores características de TCP y UDP. Es orientado a *mensajes* (como UDP) pero *fiable y ordenado* (como TCP).⁹ Sus innovaciones clave son el *multi-streaming* y el *multi-homing*.⁹ El multi-streaming permite que múltiples flujos de datos independientes existan dentro de una única conexión SCTP, evitando el bloqueo de cabecera de línea (HOL-blocking) a nivel de aplicación (ej. un navegador puede cargar imágenes en un flujo sin ser bloqueado por la pérdida de un paquete en el flujo del HTML). El multi-homing permite que una única conexión esté asociada a múltiples direcciones IP en cada extremo, proporcionando resiliencia de red.
- **QUIC (Quick UDP Internet Connections):** Es el protocolo de transporte que impulsa HTTP/3 y está diseñado para reemplazar a TCP en la web. Está encapsulado sobre *UDP*.¹⁰ Al igual que SCTP, proporciona multiplexación de flujos para resolver el HOL-blocking.¹⁰ Sus ventajas clave son una latencia de establecimiento de conexión drásticamente reducida (incluyendo O-RTT)¹² y un control de congestión más ágil y evolucionado.¹¹

Ubicación en el Sistema: Kernel vs. Espacio de Usuario:

- **Kernel:** Tradicionalmente, la implementación completa de la pila de red (TCP, UDP, IP) reside en el *kernel* (espacio de kernel) del sistema operativo.¹³ Esto garantiza un alto rendimiento y un acceso seguro y protegido al hardware de red. Las aplicaciones en el espacio de usuario acceden a estas funciones mediante *llamadas al sistema* (system calls), como las definidas por la API de Sockets (ej. connect, send, recv).¹³
- **Espacio de Usuario:** La implementación de protocolos en el kernel tiene una desventaja fundamental: es extremadamente lenta de actualizar, ya que requiere una actualización

del sistema operativo. Esta es la razón principal por la que QUIC se implementa en *espacio de usuario*.¹² Al construirse sobre el simple socket UDP (que sí está en el kernel), bibliotecas de QUIC (como las de Google Chrome o Firefox) pueden implementar toda la lógica de transporte (fiabilidad, control de congestión, etc.) como parte de la propia aplicación. Esto permite una innovación y despliegue de nuevos algoritmos a la velocidad del desarrollo de software (semanas), en lugar de la velocidad de despliegue de un SO (años).

1.2. Interacción con la Capa de Red (Pregunta 2)

Asunciones de TCP y UDP sobre IP:

Tanto TCP como UDP se implementan sobre la Capa de Red (Internetworking), cuyo protocolo en el stack TCP/IP es IP (Internet Protocol). Ambos protocolos de transporte asumen el servicio más básico y "primitivo" posible de IP: un servicio de entrega de datagramas no fiable y best-effort.¹⁵ IP no ofrece ninguna garantía sobre la entrega, el orden, la ausencia de duplicados o la integridad de los datos.

Este diseño sigue el "principio de extremo a extremo": la red (IP) es deliberadamente simple y "tonta", y toda la inteligencia (fiabilidad, orden, control de flujo) debe ser implementada en los *extremos* del sistema.

- **TCP** asume la no fiabilidad de IP y *construye* la fiabilidad desde cero (usando números de secuencia, ACKs y temporizadores).
- **UDP** asume la no fiabilidad de IP y simplemente *expone* esa no fiabilidad a la aplicación, que debe manejarla si lo considera necesario.⁸

Campo de Multiplexación:

El campo en el datagrama IP que se utiliza para diferenciar (multiplexar) a qué protocolo de transporte entregar el payload es el campo de 8 bits llamado Protocolo (en el encabezado de IPv4) o Siguiente Encabezado (en el encabezado de IPv6).

Los valores más comunes para este campo (que se pueden encontrar en `/etc/protocols`) son:

- 1 para **ICMP** (Internet Control Message Protocol)
- 6 para **TCP**
- 17 para **UDP**
- 132 para **SCTP**

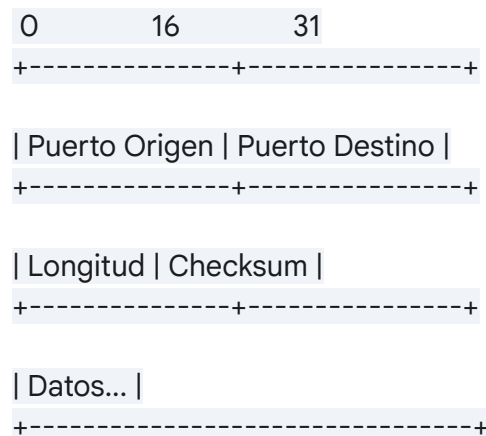
1.3. Estructuras de PDU (Pregunta 3)

La PDU (Protocol Data Unit) genérica de la capa de transporte se denomina "segmento". Sin embargo, la nomenclatura específica es:

- TCP PDU: **Segmento TCP** ⁵
- UDP PDU: **Datagrama UDP** ⁵

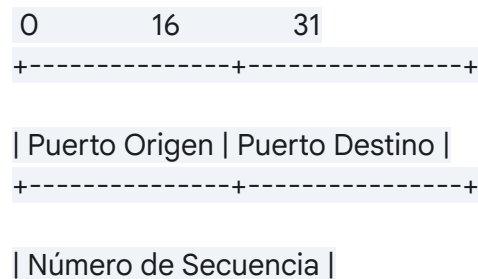
Los diagramas de sus estructuras revelan por qué sus funciones son tan diferentes.

Encabezado del Datagrama UDP (8 bytes):



- **Puerto Origen (16 bits):** Puerto del proceso emisor.
- **Puerto Destino (16 bits):** Puerto del proceso receptor.¹⁷
- **Longitud (16 bits):** Longitud total del datagrama (encabezado + datos).¹⁸
- **Checksum (16 bits):** Comprobación de errores (opcional en IPv4).¹⁸

Encabezado del Segmento TCP (20 bytes fijos + 0-40 bytes de opciones):



+-----+

| Número de Acuse de Recibo |

+-----+

| HLEN | Res | Flags | Tamaño de Ventana |

+-----+

| Checksum | Puntero Urgente |

+-----+

| Opciones (0-40 bytes)... |

+-----+

| Datos... |

+-----+

La estructura de TCP es drásticamente más compleja ¹⁹:

- **Puerto Origen/Destino (16 bits c/u):** Igual que UDP.
- **Número de Secuencia (32 bits):** Rastrea el número de byte en el flujo de datos. Esencial para el orden y la fiabilidad (ACKs).
- **Número de Acuse de Recibo (32 bits):** Indica el *próximo* byte que el emisor espera recibir. Esencial para la fiabilidad.
- **HLEN (4 bits):** Longitud del encabezado. Indica dónde empiezan los datos.
- **Flags (9 bits):** Banderas de control (ej. SYN, ACK, FIN, RST) para gestionar los estados de la conexión.¹⁹
- **Tamaño de Ventana (16 bits):** Usado para el control de flujo; indica cuánto espacio de buffer tiene disponible el receptor.¹⁸
- **Checksum (16 bits):** Comprobación de errores (siempre obligatoria).
- **Opciones (0-40 bytes):** Para negociar parámetros al inicio de la conexión, como el MSS (Maximum Segment Size), SACK (Selective ACK), y Timestamps.²¹

La diferencia en los encabezados explica directamente la diferencia en las funciones: UDP carece de los campos para números de secuencia, acuses de recibo, flags de estado o tamaño de ventana, por lo que es *físicamente incapaz* de proporcionar fiabilidad, orden, gestión de conexión o control de flujo por sí mismo.

1.4. Mecanismos de Checksum (Pregunta 5)

Existen diferencias fundamentales en cómo se calculan y utilizan los checksums:

- **Checksum IPv4:** Se encuentra en el encabezado de IPv4. Es un checksum de 16 bits que *solo cubre el encabezado de IPv4*, no los datos (payload).²² Su propósito es detectar corrupción en el encabezado (ej. en la dirección IP destino). Debe ser recalculado por cada router que modifica el TTL, lo que añade latencia de procesamiento en los routers.²²
- **Checksum IPv6:** El encabezado de IPv6 *no tiene checksum*.²² Se eliminó deliberadamente para acelerar el procesamiento de paquetes en los routers, asumiendo que las capas de enlace (L2, ej. Ethernet CRC) y transporte (L4, ej. TCP/UDP) ya proporcionan suficiente protección contra errores.
- **Checksum UDP:** Es un checksum de 16 bits que cubre el encabezado UDP, los datos UDP, y un "pseudo-encabezado" que incluye las IPs de origen/destino y el campo de protocolo. En **IPv4, el checksum de UDP es opcional** (puede ser 0). En **IPv6, el checksum de UDP es obligatorio**.
- **Checksum TCP:** Es un checksum de 16 bits que cubre el encabezado TCP, los datos TCP y el mismo "pseudo-encabezado". En **TCP, el checksum es siempre obligatorio**.¹⁹

El "pseudo-encabezado" es una construcción de software que permite a la capa de transporte (TCP/UDP) verificar que el paquete no solo está internamente intacto, sino que también ha sido entregado por IP al *host correcto* y al *protocolo correcto*.

La obligatoriedad del checksum de UDP en IPv6 es una consecuencia directa de la eliminación del checksum en el encabezado de IPv6. Sin un checksum en L3, la única protección contra un paquete entregado al host incorrecto (debido a corrupción de la IP destino) recae en la L4. TCP siempre tuvo esta protección; UDP tuvo que hacerla obligatoria para compensar la falta de protección en IPv6.

Sección 2: Gestión de Conexión y Señalización de Errores

Esta sección aborda las preguntas 4, 6, 12 y 13¹, cubriendo el inicio, la multiplexación, la transferencia de datos y la señalización de errores en las conexiones.

2.1. Señalización de Errores y Rechazos (Pregunta 4)

Las respuestas del host a paquetes no deseados o malformados son cruciales para el

diagnóstico de la red:

- **a) Datagrama IPv6 a un host sin IPv6:** El host no reconocerá el EtherType (0x86DD) en la trama de la capa de enlace (Ethernet). La trama será descartada silenciosamente por la tarjeta de red (NIC) o por la capa de enlace del sistema operativo.
- **b) Segmento TCP a un host sin TCP:** El stack IP del host recibirá el datagrama. Leerá el campo "Protocolo" (valor 6) en el encabezado IP. Al no encontrar un manejador o stack de protocolo para el protocolo 6, el sistema operativo generará y enviará un mensaje **ICMP "Destination Unreachable" (Tipo 3), Code 2 "Protocol Unreachable"** de vuelta al emisor.
- **c) Segmento TCP a un puerto cerrado:** El stack TCP del host recibe el segmento (usualmente un SYN para iniciar una conexión). El SO busca en su tabla de sockets un proceso en estado LISTEN en el puerto destino indicado. Al no encontrar ninguno, el SO responde *inmediatamente* al emisor con un segmento **TCP con el flag RST (Reset) activado**.²⁵
- **d) Mensaje UDP a un puerto cerrado:** El stack UDP del host recibe el datagrama. Busca un socket escuchando en ese puerto. Al no encontrar ninguno, el SO responde generando y enviando un mensaje **ICMP "Destination Unreachable" (Tipo 3), Code 3 "Port Unreachable"**.²⁷

Esta diferencia en las respuestas (TCP-RST vs. UDP-ICMP) es la base teórica de las técnicas de escaneo de puertos de red utilizadas por herramientas como nmap.²⁹ Un escaneo SYN (-sS) que recibe un RST sabe que el puerto está "cerrado". Un escaneo UDP (-sU) que recibe un ICMP Port Unreachable sabe que el puerto está "cerrado".

La respuesta inmediata con RST también es una optimización del rendimiento²⁵: la aplicación cliente que intenta conectarse (ej. connect()) falla inmediatamente, en lugar de esperar un largo temporizador de SYN_WAIT (que puede durar minutos).²⁵

2.2. Multiplexación de Conexiones TCP (Pregunta 6)

Un sistema operativo rastrea e identifica unívocamente cada conexión TCP mediante una **5-tupla**: {Protocolo, IP Origen, Puerto Origen, IP Destino, Puerto Destino}. Para TCP, el protocolo es siempre 6.

- **a) y b) Conexiones desde 10.0.0.10:** El servidor (192.168.1.10) escucha en un puerto de servicio fijo (ej. Telnet, puerto 23³⁰).
 - Cuando el *primer* proceso en 10.0.0.10 se conecta, el SO cliente le asigna un **puerto efímero**³¹ (ej. 49152).
 - Cuando el *segundo* proceso desde el *mismo* host (10.0.0.10) se conecta al *mismo*

servicio, el SO cliente le asigna un *segundo puerto efímero, diferente* (ej. 49153).

La siguiente tabla ilustra cómo las 5-tuplas resultantes son únicas:

Tabla 1: Identificación de Conexiones (Pregunta 6a, 6b)

Conexión	Proto	IP Origen	Puerto Origen	IP Destino	Puerto Destino (Servicio)	¿Única?
1 (Q6.a)	TCP	10.0.0.10	49152	192.168.1.10	23	Sí
2 (Q6.b)	TCP	10.0.0.10	49153	192.168.1.10	23	Sí (P. Origen dif.)

- **c) Conexión desde 10.0.0.11:** (TCP, 10.0.0.11, 49154, 192.168.1.10, 23). Esta tupla es única porque la IP Origen es diferente.
- **e) Flags Transmitidos (Establecimiento):** Se utiliza el 3-way handshake ¹⁹:
 1. Cliente -> Servidor: **SYN**
 2. Servidor -> Cliente: **SYN, ACK**
 3. Cliente -> Servidor: **ACK**
- **f) Segmentos Totales (3 Conexiones, 1 byte + cierre):**
 - Establecimiento (3-way handshake): 3 segmentos (SYN, SYN-ACK, ACK).³⁴
 - Transferencia (1 byte): 2 segmentos (1. `` con el byte; 2. [ACK] de vuelta).³⁵
 - Cierre (4-way handshake): 4 segmentos (1. [FIN, ACK], 2. [ACK], 3. [FIN, ACK], 4. [ACK]).³⁴
 - Total por conexión: $3 + 2 + 4 = 9$ segmentos.
 - Total para 3 conexiones: $9 * 3 = 27$ segmentos.
- **g) Datagramas Totales (UDP):** UDP es no orientado a conexión.⁴ No hay establecimiento (SYN) ni cierre (FIN).
 - Conexión 1: 1 datagrama (con 1 byte).
 - Conexión 2: 1 datagrama (con 1 byte).
 - Conexión 3: 1 datagrama (con 1 byte).
 - Total: **3 datagramas**.

La comparación entre (f) y (g) ilustra el *overhead* de TCP. Para enviar 1 byte de datos, TCP requiere 8 paquetes adicionales (un 900% más) en comparación con UDP, sin siquiera contar el mayor tamaño de su encabezado (20+ bytes vs 8).¹⁸ Esta es la razón por la cual los servicios sensibles a la latencia que transfieren datos pequeños (como DNS, VoIP o juegos)

deben usar UDP.⁴

2.3. Escenario de Negociación TCP (Pregunta 12)

- **Datos:**
 - Host A: ISN=50430, MSS=1400, Win=64KB
 - Host B: ISN=68900, MSS=1000, Win=32KB
- **a) Intercambio de Mensajes (3-way Handshake):**³³
 1. **A -> B:** Flags=, Seq=50430, Ack=0, Win=64K, Opciones:
 2. **B -> A:** Flags=, Seq=68900, Ack=50431 (50430+1), Win=32K, Opciones:
 3. **A -> B:** Flags=[**ACK**], Seq=50431, Ack=68901 (68900+1), Win=64K

Es crucial notar que el MSS no se "negocia" (acordando un valor común), sino que se *declara*.³⁷ Cada host anuncia el tamaño máximo de segmento que *puede recibir*. El resultado es que:

- **A** debe enviar segmentos de tamaño ≤ 1000 bytes (para respetar el límite anunciado por B).
- **B** debe enviar segmentos de tamaño ≤ 1400 bytes (para respetar el límite anunciado por A).
- b) Segmentos máximos sin ACK:
Esto está limitado por la ventana de recepción del receptor (rwnd) y el MSS que el emisor debe usar.
 - **A -> B:** Límite = rwnd de B / MSS de A (limitado por B) = $32,768 \text{ bytes} / 1000 \text{ bytes/seg} = 32.768$. A puede enviar **32** segmentos de tamaño máximo.
 - **B -> A:** Límite = rwnd de A / MSS de B (limitado por A) = $65,536 \text{ bytes} / 1400 \text{ bytes/seg} = 46.81$. B puede enviar **46** segmentos de tamaño máximo.
 - **Con Control de Congestión:** La respuesta anterior ignora el control de congestión. En una conexión real, el envío está limitado por $\min(\text{rwnd}, \text{cwnd})$ (donde cwnd es la ventana de congestión). El cwnd inicial (IW) es pequeño (históricamente 1 MSS, hoy en día ≈ 10 MSS).³⁸ Por lo tanto, A *inicialmente* solo podría enviar ≈ 10 segmentos ($10 * 1\text{KB} = 10\text{KB}$), que es mucho menor que el rwnd de 32KB. A tendría que esperar los ACKs para aumentar su cwnd (vía Slow Start ³⁹) antes de poder llenar la ventana de recepción de B.

2.4. Escenario de Transferencia TCP (Pregunta 13)

- **Datos:** B ya recibió correctamente hasta el byte 225. A (puerto 1986) se conecta a B

- (puerto 22).
- **a) ACK de B:** B ha recibido todos los bytes desde 0 hasta 225. El *próximo byte que espera* es el 226.³⁵
 - Valor de ACK: **226**.
 - Puertos: El paquete de ACK de B a A usará: Puerto Origen=**22**, Puerto Destino=**1986**.
 - **b) Números de Secuencia de A:** A envía dos segmentos (100B, 120B). Su próximo byte a enviar es el 226 (el que B está esperando).
 - Segmento 1 (100B): Seq = **226**. (Contiene los bytes 226-325).
 - Segmento 2 (120B): Seq = 226 + 100 = **326**. (Contiene los bytes 326-445).
 - **c) ACKs de B:**
 - ACK para Seg 1: B recibe 100 bytes (hasta 325). El próximo que espera es 326. **ACK = 326**.
 - ACK para Seg 2: B recibe 120 bytes (hasta 445). El próximo que espera es 446. **ACK = 446**.
 - ACK único por los dos: TCP usa ACKs acumulativos. Si B recibe ambos segmentos antes de enviar un ACK, solo necesita enviar el último ACK (el más alto): **ACK = 446**.
 - **d) Segmento 2 (Seq 326) llega antes que el Segmento 1 (Seq 226):**
 - B recibe un segmento fuera de orden. B *aún* está esperando el byte 226.
 - En respuesta a la recepción del segmento 1 (que llega tarde), B enviará un ACK. El valor de ese ACK será el próximo byte que espera *en secuencia*.
 - Si B soporta SACK (Selective ACK) ²¹, habrá almacenado el Seg 2 en un búfer. Cuando Seg 1 (Seq 226) finalmente llega, B puede procesar inmediatamente Seg 1 y Seg 2. El ACK enviado será **446**.
 - Si B no soporta SACK (implementación antigua), B descartó Seg 2 cuando llegó fuera de orden. Cuando Seg 1 llega, lo procesa y envía un ACK por él: **ACK = 326**.
 - *Interpretación más probable (sin SACK):* El valor del ACK para el *primer segmento recibido* (el de 100 bytes, Seq 226) es **326**.

Sección 3: Diagnóstico de Estado de Red con netstat

Esta sección resuelve las preguntas 7, 9 y 10, que requieren un análisis experto de las salidas de la herramienta netstat.¹

3.1. Análisis de Estado netstat -atun (Pregunta 7)

A continuación se presenta un análisis detallado de la salida del comando netstat de la página

2 de.¹¹

- a) ¿Cuántas conexiones TCP hay establecidas?
Contando las líneas con Proto=tcp y State=ESTABLISHED, hay 8 líneas. (Nota: hay una línea duplicada, 127.0.0.1:45547, por lo que el número de conexiones únicas es 7).
- b) **¿Cuántas como cliente y cuántas como servidor?**
 - **Cliente:** Conexiones iniciadas desde el host local (usando un puerto local efímero/alto) hacia un servicio (puerto foráneo bajo/fijo). Hay **5** conexiones de cliente externas (una a puerto 7, cuatro a puerto 443).
 - **Servidor:** Conexiones hacia un servicio local que está en LISTEN.
 - 127.0.0.1:8000 -> 127.0.0.1:35250: El servicio en el puerto 8000 (en LISTEN) está actuando como servidor.
 - 127.0.0.1:45547 -> 127.0.0.1:43695: El servicio en el puerto 43695 (en LISTEN) está actuando como servidor.Hay 2 conexiones de servidor (ambas loopback).
- c) ¿Cuántas conexiones TCP están pendientes por establecerse?
Se busca el estado SYN_SENT (intento de conexión activa) o SYN_RECV (esperando el ACK final del handshake). Hay 1 conexión en estado SYN_SENT.
- d) ¿A qué servicios bien conocidos (0-1023) tiene conexiones TCP establecidas?
Se buscan puertos foráneos < 1024 en estado ESTABLISHED.
 - Puerto **443** (HTTPS)³⁰
 - Puerto **7** (ECHO)
- e) ¿Qué servicios bien conocidos (TCP y UDP) tiene corriendo (esperando)?
Se buscan puertos locales < 1024 en estado LISTEN (TCP) o listados (UDP).
 - **TCP:** Puerto **22** (SSH), Puerto **631** (IPP/CUPS), Puerto **80** (HTTP).³⁰
 - **UDP:** Puerto **68** (DHCP/BOOTP Client), Puerto **69** (TFTP).
- f) ¿Cuántas conexiones están en proceso de cierre?
Se buscan todos los estados de cierre 41:
 - CLOSE_WAIT: 2
 - TIME_WAIT: 1
 - LAST_ACK: 1
 - Total: **4** conexiones.
- g) **¿Cierre iniciado por host local o remoto?**
 - **Iniciado por Remoto (Cierre Pasivo):** El host local está en CLOSE_WAIT o LAST_ACK.⁴¹ Esto ocurre cuando el otro extremo envió un FIN primero.
 - Total Remoto: 2 (CLOSE_WAIT) + 1 (LAST_ACK) = **3** conexiones.
 - **Iniciado por Local (Cierre Activo):** El host local está en FIN_WAIT_1, FIN_WAIT_2 o TIME_WAIT.⁴¹ Esto ocurre cuando la aplicación local inició el cierre.
 - Total Local: 1 (TIME_WAIT) = **1** conexión.

Es importante notar que la presencia de múltiples sockets en estado CLOSE_WAIT (Pregunta 7.f) no es un estado de red transitorio normal, sino un fuerte indicio de un *bug* de software o una fuga de recursos (resource leak).⁴³ El estado CLOSE_WAIT significa que el kernel local ha

recibido el FIN del par remoto, pero la aplicación local (en los puertos 50120 y 58433) aún no ha ejecutado la llamada close() en su socket. Estos sockets "zombie" siguen consumiendo recursos del sistema.

- h) ¿En qué puertos podría recibir datos sobre IPv6 desde otros hosts?
Se buscan puertos tcp6 (LISTEN) y udp6 que escuchen en la dirección "any" (:::), no en la dirección loopback (::1).
 - :::80 (TCP6)
 - :::22 (TCP6)
 - :::59695 (UDP6)
 - :::5353 (UDP6)

3.2. Análisis de Procesos netstat -atunp (Pregunta 9)

Análisis de la salida de netstat -atunp.¹¹ La opción -p añade la columna PID/Nombre de Proceso.⁴⁴

- a) Proceso de Impresión: El puerto de servicios de impresión de red es el 631 (IPP/CUPS). La línea correspondiente es:
127.0.0.1:631... LISTEN... 11351/cupsd
El proceso es cupsd con PID 11351.
- b) ¿Imprimir desde la red?
No. La columna "Local Address" muestra 127.0.0.1:631. Esta es la dirección de loopback (localhost).⁴⁶ El servicio cupsd solo está aceptando conexiones que se originan desde el mismo host. Para aceptar conexiones de red, debería escuchar en 0.0.0.0:631 (todas las interfaces IPv4) o en la IP específica de la LAN.
- c) ¿Qué servicios pueden recibir conexiones sobre IPv6?
Se buscan las líneas tcp6 en estado LISTEN:
 - **Puerto 80** (:::80), Proceso: 1847/apache2
 - **Puerto 22** (:::22), Proceso: 999/sshd
 - **Puerto 631** (::1:631), Proceso: 11351/cupsd (Este solo acepta conexiones IPv6 desde localhost).

3.3. Diagnóstico de Falla de Conexión (Pregunta 10)

Análisis de las salidas de netstat del servidor (srv) y cliente (cli).¹⁴¹

- **Datos:**

- srv: 157.0.0.1:4500... 157.0.11.1:52843 SYN_RECV
- cli: 157.0.11.1:52843... 157.0.0.1:4500 SYN_SENT
- a) Paquetes Perdidos:
Se reconstruye el 3-way handshake mentalmente:
 1. cli envía `` a srv. (El estado de cli cambia a SYN_SENT).
 2. srv recibe el , envía a cli. (El estado de srv cambia a SYN_RECV).
 3. cli recibe el `` , envía [ACK] a srv. (Ambos estados cambian a ESTABLISHED).

Análisis del Problema: El cliente (cli) está atascado en el estado SYN_SENT.⁴⁷ El servidor (srv) está atascado en el estado SYN_RECV.⁴⁷ Conclusión: El del cliente (Paso 1) ****Sí**** llegó al servidor (porque el servidor pasó de `LISTEN` a `SYN\_RECV`). Sin embargo, el del servidor (Paso 2) NO llegó al cliente (porque el cliente sigue en SYN_SENT, esperando ese paquete). Paquete perdido: El paquete `` en la ruta de srv a cli.
- b) Estado Final:
Si la conexión no se establece, ambos lados sufrirán un timeout de sus estados transitorios.
 - srv (en SYN_RECV) finalmente descartará la conexión medio-abierta y volverá al estado LISTEN, listo para una nueva conexión.⁴⁷
 - cli (en SYN_SENT) retransmitirá el `` varias veces. Tras el *timeout* final, la llamada connect() de la aplicación fallará, típicamente con un error "Connection timed out".²⁵

Sección 4: Análisis Forense de Trazas de Red (Packet-Level)

Esta sección resuelve las preguntas 8, 14, 15 y 19¹, que requieren un análisis a nivel de paquete de capturas de red (Wireshark).

4.1. Deducción de Campos Ocultos (Pregunta 8, Figura 1)

Análisis de la captura del 3-way handshake en Figura 1.¹¹

- **Puerto [blurred]:** El nombre simbólico del puerto vce (visto en las líneas 1, 2 y 3) se mapea explícitamente a **11111** en el desglose detallado de la línea 3.
- **Seq (Línea 1) [blurred]:** El paquete de Línea 2 (SYN, ACK) tiene un Ack=3933822138. En un SYN-ACK, el número de ACK es siempre ISN_del_cliente + 1.³⁵ Por lo tanto, el Número de Secuencia Inicial (ISN) del cliente (Línea 1) debe ser $3933822138 - 1 = \mathbf{3933822137}$.
- **Stream Index [blurred]:** El desglose de la Línea 3 muestra explícitamente (Stream index:

0). El valor es 0.

- **Flags (Detalle) [blurred]:**

- Syfil Set: Basado en el contexto y el valor de Flags: 0x012 (SYN, ACK), este campo debe ser **SYN: Set**.
- Fin: Nat set: Debe ser **FIN: Not set**.

4.2. Reconstrucción de Sesión TCP (Pregunta 14, Figura 3)

Se completan todos los valores ? en la traza de la Figura 3 ¹, siguiendo la lógica de seguimiento de bytes de TCP, donde Seq es el número del primer byte enviado y Ack es el próximo byte esperado.³⁵

Tabla 2: Reconstrucción de la Sesión TCP de la Figura 3

Paquete	Comentario	Seq (Corregido)	Ack (Corregido)	Razón de la Corrección (Lógica de Seguimiento de Bytes)
1	SYN (10.0.0.10)	0	-	ISN del Cliente = 0.
2	SYN, ACK (10.0.1.10)	0	1	ISN del Servidor = 0. Ack = ISN_cli + 1.
3	ACK (10.0.0.10)	1	1	Cliente confirma el handshake. Seq=ISN_cli+1. Ack=ISN_srv+1.
4	PSH, ACK (Len 7)	1	1	Cliente envía 7 bytes (bytes 1 al 7).

5	ACK (10.0.1.10)	1	8	Servidor acusa recibo de los 7 bytes. Próximo byte esperado: $1+7 = 8\$$.
6	PSH, ACK (Len 9)	8	1	Cliente envía 9 bytes (bytes 8 al 16).
7	ACK (10.0.1.10)	1	17	Servidor acusa recibo de los 9 bytes. Próximo byte esperado: $8+9 = 17\$$.
8	PSH, ACK (Len 5)	17	1	Cliente envía 5 bytes (bytes 17 al 21).
9	ACK (10.0.1.10)	1	22	Servidor acusa recibo de los 5 bytes. Próximo byte esperado: $17+5 = 22\$$.
10	FIN, ACK (10.0.0.10)	22	1	Cliente inicia el cierre. El flag FIN consume 1 número de secuencia. Seq=22 (último byte 21 + 1).
11	FIN, ACK (10.0.1.10)	1	23	Servidor acusa recibo del FIN del cliente. ³⁶ Próximo esperado: $22+1 = 23\$$.

				También envía su propio FIN (Seq=1, ya que no envió datos).
12	ACK (10.0.0.10)	23	2	Cliente acusa recibo del FIN del servidor. Seq=22+1=23. Próximo esperado: $1+1 = 2$. Cierre completo.

4.3. Análisis de Opciones TCP (Pregunta 15, tcp*.pcap)

Esta pregunta requiere el uso de un analizador de protocolos como Wireshark para inspeccionar los archivos pcap.⁴⁹

Proceso de Análisis:

1. Abrir tcp-init.pcap y tcp-init2.pcap en Wireshark.
2. Aplicar el filtro tcp.flags.syn == 1 para aislar los paquetes de establecimiento de conexión.
3. **Paquete `` (Cliente -> Servidor):**
 - Seleccionar el primer paquete.
 - En el panel "Packet Details", expandir "Transmission Control Protocol".
 - Registrar: Source Port (puerto del cliente), Destination Port (puerto del servidor).
 - Registrar: Sequence Number (raw) (Este es el **ISN** del cliente).
 - Expandir la sección "Options":
 - Registrar TCP Option - Maximum segment size: XXXX bytes (Este es el **MSS** del cliente).²¹
 - Verificar la presencia de TCP Option - SACK permitted (SACK).²¹
4. **Paquete `` (Servidor -> Cliente):**
 - Repetir el paso 3 para obtener el **ISN** del servidor, su **MSS** y su soporte de **SACK**.

Comparación con tcp-init3.pcap (Diferencia de MSS):

Al repetir el proceso para tcp-init3.pcap, se observará un valor de MSS diferente (probablemente más bajo).

- **Causa:** El MSS (Maximum Segment Size) no es un valor fijo; se calcula en base al MTU (Maximum Transmission Unit) de la interfaz de salida, típicamente como $MSS = MTU - 40$ bytes (20 para encabezado IP, 20 para TCP).⁵⁰
- Un MSS más bajo indica un MTU de ruta más bajo. La causa más común de esto es la **encapsulación de red**. Es muy probable que la traza tcp-init3.pcap haya sido capturada en una ruta que atraviesa un **túnel VPN** (como IPsec) o un **túnel GRE**.⁵¹ Estos túneles añaden sus propios encabezados al paquete IP original, reduciendo el espacio disponible para el payload de TCP y forzando al SO a anunciar un MSS más pequeño para evitar la fragmentación.

4.4. Flujo vs. Congestión (Pregunta 19, tcp*.pcap.gz)

Diferenciar entre el control de flujo y el control de congestión en una captura de Wireshark es clave para el diagnóstico de rendimiento.

1. Control de Flujo (Flow Control):

- **Causa:** El *receptor* está lento o su buffer de aplicación está lleno. No puede procesar los datos lo suficientemente rápido.
- **Mecanismo:** El receptor anuncia un **rwnd (Receive Window)** más pequeño, o incluso 0.
- **Indicadores en Wireshark:**
 - **TCP ZeroWindow**⁵²: El receptor anuncia Window Size: 0. El emisor debe detenerse legalmente.
 - **TCP Window Full**⁵³: El emisor ha enviado una cantidad de datos igual a la última rwnd anunciada y ahora está (correctamente) detenido, esperando un TCP Window Update (un ACK con un tamaño de ventana > 0).
- **Diagnóstico:** El problema es el *receptor*.

2. Control de Congestión (Congestion Control):

- **Causa:** La *red* intermedia está congestionada (routers) y está descartando paquetes.
- **Mecanismo:** El *emisor* infiere la pérdida (por 3 ACKs duplicados o un RTO) y reduce su **cwnd (Congestion Window)** para aliviar la red.
- **Indicadores en Wireshark:**
 - **TCP Duplicate ACK**⁵⁵: El receptor notifica que ha recibido paquetes fuera de orden (un síntoma de pérdida).
 - **TCP Fast Retransmission**⁵⁶: El emisor reacciona a 3 ACKs duplicados retransmitiendo el paquete perdido *antes* de que expire el RTO.
- **Diagnóstico:** El problema es la *red*.

La señal definitiva es la **correlación**: si se observan retransmisiones (Fast Retransmission)

mientras que los ACKs del receptor siguen mostrando una rwnd grande (Wireshark calcula esto como [Calculated window size]), el problema es inequívocamente el control de congestión.

Sección 5: Rendimiento, Fiabilidad y Control de Congestión

Esta sección resuelve las preguntas 11, 16, 17 y 18¹, centrándose en el análisis de rendimiento y los algoritmos que gestionan la fiabilidad.

5.1. Análisis de Go-Back-N (Pregunta 11, Figura 2)

Análisis del diagrama de secuencia de Figura 2¹ bajo el protocolo ARQ Go-Back-N.⁵⁷

- **Datos del Diagrama:**
 - Tamaño de Paquete (Data) = 512 bytes.
 - Delay (Tx → Rx) = 0.0105 s.
 - Delay (Rx → Tx) = 0.004 s.
 - Round-Trip Time (RTT) = $0.0105 + 0.004 = 0.0145$ s.
- a) Ancho de Banda (Throughput):

El throughput (ancho de banda digital) es el número de bits útiles transferidos por segundo. Para calcular el throughput óptimo del enlace, primero debemos determinar la ventana óptima (ver punto c).

 - El diagrama sugiere una ventana de al menos 2 (Tx envía 1002 y 1514 antes de esperar). Asumiendo una ventana óptima de $W=2$:
 - Bytes por RTT = $2 \text{ paquetes} * 512 \text{ bytes/paquete} = 1024 \text{ bytes}$.
 - Throughput = $\frac{\text{Bytes}}{\text{Tiempo}} = \frac{1024 \text{ bytes}}{0.0145 \text{ s}} \approx 70,620 \text{ B/s}$ (o 70.6 KB/s).
- b) **Valor Óptimo del T1 (RTO):**
 - Si el delay es constante, un RTO estático óptimo debe ser ligeramente mayor que el RTT para evitar retransmisiones espurias. Una regla común es $RTO \approx 2 * RTT$.⁵⁸
 - $RTO = 2 * 0.0145 \text{ s} = \mathbf{0.029 \text{ s}}$.
 - **Si el delay no es constante:** Un RTO estático es ineficiente. Se debe usar un algoritmo dinámico como el de **Jacobson-Karn**.⁵⁹ Este algoritmo mantiene un RTT suavizado (SRTT) y una desviación media del RTT (RTTVAR). El RTO se calcula

dinámicamente como $RTO = SRTT + 4 * RTTVAR$ ⁵⁹, adaptándose a la variabilidad de la red.

- c) Tamaño de Ventana Óptimo:
El tamaño de ventana óptimo (W) es aquel que permite al emisor transmitir continuamente, llenando el "conducto" de la red. Se calcula como el Producto Ancho de Banda-Retardo (BDP).
 - $BDP = \text{Throughput} * RTT$
 - $BDP = (70,620 \text{ B/s}) * (0.0145 \text{ s}) = 1024 \text{ bytes}$.
 - En paquetes: $W = BDP / \text{Tamaño de Paquete} = 1024 \text{ bytes} / 512 \text{ bytes/paquete} = \mathbf{2 \text{ paquetes}}$.
- d) Bytes Efectivos Transferidos:
En el escenario del diagrama, los primeros dos paquetes (1002, 1514) se envían, sus ACKs se pierden, el RTO expira, y se retransmiten 3 paquetes (1002, 1514, 2026). Asumiendo que estos 3 últimos son recibidos y confirmados, los bytes efectivos transferidos (sin contar retransmisiones) son $3 * 512 = 1536 \text{ bytes}$.
- e) Completar ? (Primer ACK se pierde):
 - Tx envía data #seq 1002.
 - Tx envía data #seq 1514 (asumiendo W=2).
 - Rx recibe 1002. Envía ACK=???? -> Próximo esperado: $1002+512=1514$. **ACK = 1514**. (Este se PIERDE).
 - Rx recibe 1514. Envía ACK=???? -> Próximo esperado: $1514+512=2026$. **ACK = 2026**. (Este también debe perderse para que RTO(1002) expire).
 - RTO(1002) expira. Tx Goes Back.⁵⁷
 - Tx re-envía data #seq=1002...
 - Tx re-envía data #seq=????... -> **seq=1514**.
 - Tx re-envía data #seq=????... -> **seq=2026**.
 - Rx recibe 1002 (duplicado), lo descarta, envía ACK=2026 (sigue esperando 2026).
 - Rx recibe 1514 (duplicado), lo descarta, envía ACK=2026.
 - Rx recibe 2026 (¡nuevo!). Envía ACK=???? -> Próximo esperado: $2026+512=\mathbf{2538}$.

5.2. Crecimiento de Ventana (Pregunta 16)

- **Datos:** RTT=10ms, rwnd=12KB, cwnd_inicial=1KB (1xMSS), ssthresh_inicial=16KB, MSS=1KB.
- La ventana utilizada está limitada por $\min(cwnd, rwnd)$. El máximo alcanzable es rwnd = 12KB.
- Actualmente, cwnd (1KB) es menor que ssthresh (16KB), por lo tanto, la conexión está en la fase de **Slow Start**.³⁹
- En Slow Start, cwnd se duplica cada RTT.³⁹

Tabla 3: Evolución de Slow Start (Pregunta 16)

Tiempo (Acum.)	RTT #	Evento	cwnd	ssthresh	Ventana Usada min(cwnd, rwnd)
0ms	0	Inicio	1KB	16KB	1KB
10ms	1	ACK(s) recibido(s)	2KB	16KB	2KB
20ms	2	ACK(s) recibido(s)	4KB	16KB	4KB
30ms	3	ACK(s) recibido(s)	8KB	16KB	8KB
40ms	4	ACK(s) recibido(s)	16KB	16KB	12KB (tope de rwnd)

En T=40ms (RTT 4), cwnd se duplica a 16KB. En este punto, cwnd alcanza ssthresh, y la fase cambiaría a Congestion Avoidance. Sin embargo, la ventana utilizada (min(cwnd, rwnd)) se convierte en $\min(16KB, 12KB) = 12KB$.

La ventana utilizada alcanza su máximo (12KB) en 40ms.

5.3. Reacción a RTO (Pregunta 17)

- **Datos:** cwnd=18KB, MSS=1KB. Evento: **RTO Timeout** (vencimiento del temporizador de retransmisión).
- La reacción de TCP (Tahoe y Reno) a un RTO Timeout es la señal más severa de congestión y la reacción es drástica⁶³:
 1. El ssthresh (umbral) se fija en $cwnd / 2 = 18KB / 2 = 9KB$.
 2. El cwnd (ventana de congestión) se resetea a 1 MSS = **1KB**.
 3. La conexión re-entra en la fase de **Slow Start**.
- **Evolución en las próximas 4 ráfagas (RTTs) exitosas:**
 - Ráfaga 1 (post-RTO): cwnd (1KB) < ssthresh (9KB). cwnd se duplica a **2KB**.

- Ráfaga 2: $\text{cwnd} (2\text{KB}) < \text{ssthresh} (9\text{KB})$. cwnd se duplica a **4KB**.
- Ráfaga 3: $\text{cwnd} (4\text{KB}) < \text{ssthresh} (9\text{KB})$. cwnd se duplica a **8KB**.
- Ráfaga 4: $\text{cwnd} (8\text{KB}) < \text{ssthresh} (9\text{KB})$. cwnd se duplica a 16KB. Sin embargo, al *alcanzar o superar* el ssthresh , debe cambiar a Congestion Avoidance.³⁹ En la práctica, cwnd crecerá a **9KB** (el valor de ssthresh) y en la *siguiente* ráfaga (la 5ta) crecerá linealmente a 10KB.

La ventana será de **9KB** (o 9 segmentos) después de las 4 ráfagas, momento en el que pasa de Slow Start a Congestion Avoidance.

5.4. Análisis de TCP Reno (Pregunta 18, Figura 4)

Análisis del gráfico de Figura 4 (TCP Reno)¹, que muestra la evolución de cwnd (en segmentos).⁶⁶

- **a) Intervalos de Slow-Start:** Se busca un crecimiento *exponencial* (curva cóncava hacia arriba).
 - : cwnd crece de 1 a 32 (1, 2, 4, 8, 16, 32).
 - : cwnd crece de 1 a 8 (1, 2, 4, 8).
- **b) Intervalos de Congestion Avoidance:** Se busca un crecimiento *lineal* (línea recta, +1 MSS por RTT).³⁹
 - : cwnd crece linealmente de 32 a ≈ 37 .
 - : cwnd crece linealmente de ≈ 18 a 26.
 - : cwnd crece linealmente de 13 a 17.
 - : cwnd crece linealmente de 8 a 10.
- c) Evento en $R=22$:
 En la ronda 22, cwnd cae abruptamente de 17 a 1. Esta reacción (resetear cwnd a 1 MSS) es la respuesta a un RTO Timeout.⁶³
 Esto contrasta con los eventos en $R=10$ y $R=18$. En esos puntos, cwnd se reduce a la mitad ($37 \rightarrow 18$; $26 \rightarrow 13$). Esa reacción (reducir a $\text{cwnd}/2$) es la respuesta de Reno a 3 ACKs Duplicados (Fast Retransmit).⁷
- **d) Valor de SSThreshold:** ssthresh es el valor al que cwnd se reduce (en 3-Dup-ACK) o el valor donde Slow Start se detiene.
 - **Inicialmente (en $R=0$):** El primer Slow Start se detiene al alcanzar $\text{cwnd}=32$. ssthresh inicial = **32**.
 - **En $R=18$:** Ocurre un 3-Dup-ACK. cwnd estaba en 26. El nuevo ssthresh se fija en $\text{cwnd} / 2 = 26 / 2 = \mathbf{13}$. (El gráfico lo confirma, ya que el siguiente crecimiento lineal [18-22] comienza desde 13).
 - **En $R=24$:** Ocurre un RTO en $R=22$. cwnd estaba en 17. El nuevo ssthresh se fija en $\text{cwnd} / 2 = 17 / 2 \approx \mathbf{8}$. (El gráfico lo confirma, ya que el Slow Start [22-24]

se detiene al alcanzar $cwnd=8$ y pasa a Congestion Avoidance).

Sección 6: Escenarios de Traducción de Direcciones de Red (NAT)

Esta sección resuelve las preguntas 20 y 21¹, que tratan sobre NAT Básico y NAPT (Overloading).

6.1. Construcción de Tabla NAPT (Pregunta 20, Figura 5)

- **Datos:** Figura 5.¹ El router n1 (con IP pública única 200.16.5.9) debe hacer "overloading" (NAPT) para dos conexiones TCP.
- **Conexiones:**
 1. n2 (192.168.1.69) -> s4 (163.10.45.34)
 2. n3 (192.168.1.80) -> s4 (163.10.45.34)
- **NAPT (Network Address Port Translation):** También conocido como PAT o Masquerading.⁶⁷ El router n1 debe mapear ambas conexiones a su *única* IP pública (200.16.5.9). Para diferenciar las respuestas, debe asignar puertos de origen únicos en el lado público.

Tabla 4: Tabla de Traducción NAPT en Router n1 (Pregunta 20)

Origen (Privado)	Origen (Público Traducido)	Destino (Público)
192.168.1.69 : 50001 (Puerto efímero de n2)	200.16.5.9 : 62001 (Puerto traducido 1)	163.10.45.34 : P_Servidor
192.168.1.80 : 50002 (Puerto efímero de n3)	200.16.5.9 : 62002 (Puerto traducido 2)	163.10.45.34 : P_Servidor

Cuando s4 responda a 200.16.5.9:62001, n1 lo traducirá de vuelta a 192.168.1.69:50001.

6.2. Detección de NAT e Hipótesis (Pregunta 21)

- **Indicios de NAT en Ejercicio 7:** Sí, los indicios son claros. La tabla de netstat ¹ muestra múltiples conexiones ESTABLISHED donde la "Local Address" es 10.168.1.163. Esta es una dirección IP privada del bloque 10.0.0.0/8 (RFC 1918). La "Foreign Address" es, por ejemplo, 173.194.42.54, una dirección IP pública.
- Es *imposible* que un paquete con una IP de origen privada sea enrutado en la Internet pública; los routers de borde la descartan. Por lo tanto, *debe* existir un dispositivo (router) entre el host 10.168.1.163 y el host 173.194.42.54 que esté realizando NAT/NAPT.⁶⁷ La salida de netstat en el host local no es consciente de esta traducción; solo muestra el estado del socket local.⁷¹
- **Conexiones para Tablas:** Las "tres primeras conexiones de salida en estado ESTABLISHED" (no-loopback) de Q7 son:
 1. 10.168.1.163:51222 -> 173.194.42.54:443
 2. 10.168.1.163:36121 -> 138.160.162.116:7
 3. 10.168.1.163:60123 -> 91.189.89.76:443
- **Tabla 5: Hipótesis NAT Básico (1:1) (Pregunta 21)**
 - NAT Básico (Static NAT) mapea 1 IP privada a 1 IP pública, usualmente conservando los puertos.⁶⁷ Asumamos que el router administra 200.1.1.10 para este host.

Origen (Privado)	Origen (Público Traducido)	Destino (Público)
10.168.1.163:51222	200.1.1.10 : 51222	173.194.42.54:443
10.168.1.163:36121	200.1.1.10 : 36121	138.160.162.116:7
10.168.1.163:60123	200.1.1.10 : 60123	91.189.89.76:443

- **Tabla 6: Hipótesis NAPT (Overloading/Masquerading) (Pregunta 21)**
 - NAPT (Overloading) mapea *múltiples* IPs privadas a *una* IP pública.⁶⁷ Asumamos que el router administra la IP 200.1.1.1.

Origen (Privado)	Origen (Público Traducido)	Destino (Público)
10.168.1.163:51222	200.1.1.1 : 51222	173.194.42.54:443
10.168.1.163:36121	200.1.1.1 : 36121	138.160.162.116:7

10.168.1.163:60123	200.1.1.1 : 60123	91.189.89.76:443
--------------------	-------------------	------------------

En este escenario específico, las tablas de NAT Básico y NAPT se ven idénticas. NAPT ⁶⁷ solo *modifica* el puerto de origen si es necesario para evitar una colisión.⁶⁷ Dado que las tres conexiones se originan desde la *misma* IP privada (10.168.1.163), pero todas usan puertos de origen *diferentes* (51222, 36121, 60123), no hay colisión de puertos. El router NAPT puede simplemente reutilizar los mismos números de puerto. Las tablas diferirían si un *segundo host* (ej. 10.168.1.164) intentara usar el puerto 51222; el router NAPT tendría que remapear ese puerto (ej. a 51223) para evitar la colisión en la IP pública 200.1.1.1.

Sección 7: Implementación Práctica: Sockets y Herramientas

Esta sección resuelve las preguntas 22 y 23 ¹, que son ejercicios prácticos de laboratorio usando netcat, inetd y programación de sockets.

7.1. Ejercicios UDP (Pregunta 22)

- **a) nc UDP y Estado ESTABLISHED:**
 - **Servidor (n9):** nc -u -l 9000 (Escuchar en modo UDP en el puerto 9000).⁷³
 - **Cliente (n13):** echo "datos" | nc -u n9 9000 (Enviar datos usando UDP).⁷⁵
 - **Significado de ESTABLISHED en UDP:** UDP es un protocolo sin conexión ⁷⁶, por lo que este estado parece una contradicción. Sin embargo, netstat ⁷⁷ mostrará un socket UDP como ESTABLISHED si la aplicación ha usado la llamada al sistema connect() en ese socket.⁷⁸
 - Para un socket UDP, connect() no envía ningún paquete (a diferencia del SYN de TCP). Lo que hace es asociar el socket en el *kernel del cliente* con una dirección de destino fija (n9:9000). Esto tiene dos efectos:
 - 1. El cliente puede usar send() en lugar de sendto().
 - 2. El kernel del cliente *solo* aceptará datagramas en este socket que provengan de n9:9000.
 - Es una optimización del lado del cliente que netstat reporta como ESTABLISHED.
- **b) inetd UDP Echo:**
 - Se debe editar /etc/inetd.conf ⁷⁹ y descomentar (o añadir) la línea para el servicio

echo de tipo dgram (datagrama UDP) y internal (manejado por inetd).⁸⁰

- echo dgram udp wait root internal
- Un cliente (ej. Python ⁸²) puede probarlo: `sock.sendto(msg, (n9, 7))` y luego `sock.recvfrom()` para recibir el eco.

- **c) UDP a Puerto Cerrado y traceroute:**

- **Prueba:** `echo "test" | nc -u n9 12345` (donde 12345 es un puerto cerrado en n9).
- **Notificación:** El stack TCP/IP en n9, al no encontrar un socket escuchando en el puerto UDP 12345, notifica al emisor (n13) enviando un paquete **ICMP Tipo 3 (Destination Unreachable), Code 3 (Port Unreachable)**.²⁸
- **Uso en traceroute:** traceroute (en Linux/Cisco) explota este mecanismo.⁸⁴ Envía una serie de datagramas UDP a puertos UDP de destino muy altos (que se asumen inalcanzables).
 1. Envía el primer paquete con TTL=1. El primer router en la ruta descarta el paquete y responde con **ICMP Tipo 11 (Time Exceeded)**.
 2. Envía el segundo paquete con TTL=2. El segundo router responde con **ICMP Time Exceeded**.
 3. ...
 4. Eventualmente, un paquete (ej. con TTL=4) llega al host destino (n9). El TTL es válido, por lo que el host lo procesa.
 5. n9 ve que el datagrama UDP es para un puerto inalcanzable (ej. 33434) y responde con **ICMP Tipo 3 (Port Unreachable)**.⁸⁵
 6. Cuando traceroute recibe el "Port Unreachable" en lugar de "Time Exceeded", sabe que ha llegado al destino final.

7.2. Ejercicios TCP (Pregunta 23)

- a) Servidor TCP Discard (Socket API):

Un script simple de Python para esto implicaría 86:

1. `s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`
2. `s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` (Importante para reiniciar el servidor sin esperar el `TIME_WAIT` ⁸⁹)
3. `s.bind(("", 9000))`
4. `s.listen()`
5. `conn, addr = s.accept()`
6. En un bucle `while True:`, `data = conn.recv(1024)`. Si data está vacío (longitud 0), el cliente cerró, así que `break`. No hacer nada con data es, de hecho, "descartarlo".

- b) **inetd TCP Echo/Discard:**

- Editar `/etc/inetd.conf` ⁹⁰ para activar los servicios internos de inetd:
 - `echo stream tcp nowait root internal` ⁸⁰

- discard stream tcp nowait root internal ⁸⁰
- **c) Cliente nc o telnet:**
 - n13\$ nc n9 9 (Conexión a Discard. Todo lo que se escriba será descartado).
 - n13\$ nc n9 7 (Conexión a Echo. Todo lo que se escriba será devuelto).
 - telnet ⁹² también funciona, pero nc ⁹⁴ es una herramienta más "limpia" para este tipo de pruebas.
- **f) Conexión a Puerto Cerrado:**
 - n13\$ nc n9 12345 (donde 12345 está cerrado).
 - **Notificación:** El stack TCP/IP en n9 responde inmediatamente con un segmento TCP con el **flag RST (Reset) activado**.²⁵ nc reportará "Connection refused".
- **g, h) Cierre de Conexiones y Estados:**
 - Cuando el cliente (que hizo el cierre activo) cierra (ej. Ctrl+C en nc), su socket pasa por FIN_WAIT_1 -> FIN_WAIT_2 y finalmente se queda en **TIME_WAIT**.⁴¹
 - El servidor (cierre pasivo) pasa por CLOSE_WAIT -> LAST_ACK -> CLOSED.⁴¹
 - El estado en el que queda el que hace el cierre *activo* es **TIME_WAIT**.
 - Un cierre estándar (gracioso) requiere **4 segmentos** (FIN del cliente, ACK del servidor, FIN del servidor, ACK del cliente).³⁴
 - *Variantes de cierre:* Un cierre abrupto se puede forzar con un paquete **RST (Reset)** ⁹⁵, que cierra la conexión inmediatamente y libera los recursos sin pasar por los estados de cierre.
- **j) Socket en TIME_WAIT y 2MSL:**
 - n13\$ nc -p 55000 n9 7 (Usando un puerto origen específico 55000).
 - Escribir datos, recibir eco, cerrar nc.
 - n13\$ nc -p 55000 n9 7 (Intentar reabrir la *exacta misma* conexión inmediatamente).
 - **Estado:** El socket (n13:55000 -> n9:7) está en estado **TIME_WAIT** en el host n13.⁴¹
 - **Error:** El segundo comando fallará con un error "Address already in use".
 - **2MSL (Maximum Segment Lifetime):** El estado TIME_WAIT dura 2 * MSL. El MSL es el tiempo máximo que se asume que un segmento puede vivir en la red. El valor de 2MSL es típicamente de 60 segundos (MSL=30s) ⁹⁶ o 240 segundos (MSL=2min).⁹⁷
 - El propósito de este estado de espera es fundamental ⁹⁸:
 1. **Fiabilidad del Cierre:** Asegura que el ACK final (el último de los 4 segmentos) que se envió para el FIN del servidor no se perdió. Si se perdió, el servidor retransmitirá su FIN, y el host en TIME_WAIT puede re-enviar el ACK.
 2. **Prevención de Corrupción:** Permite que todos los paquetes de la conexión antigua (que podrían estar retrasados en la red) mueran y sean descartados. Si se permitiera a la aplicación reutilizar la tupla (n13:55000 -> n9:7) instantáneamente, un paquete de datos o un FIN retrasado de la *primera* conexión podría llegar y ser interpretado erróneamente como parte de la *segunda* conexión, corrompiendo el estado de la sesión.

1. redes-II-ing-practica_4.pdf
2. ¿Qué es el modelo OSI? — Explicación de las 7 capas del modelo OSI - Amazon AWS, fecha de acceso: noviembre 16, 2025, <https://aws.amazon.com/es/what-is/osi-model/>
3. Encapsulamiento de datos en Redes - CCNA desde Cero, fecha de acceso: noviembre 16, 2025, <https://ccnadesdecero.es/encapsulamiento-de-datos-redes/>
4. Transmission Control Protocol - Wikipedia, fecha de acceso: noviembre 16, 2025, https://en.wikipedia.org/wiki/Transmission_Control_Protocol
5. Top 10 ¿Qué son las unidades de datos de protocolo? Una guía completa - newsunn, fecha de acceso: noviembre 16, 2025, <https://www.newsun.com/es/understanding-protocol-data-unit-pdu-network-guide/>
6. Listado de puertos TCP y UDP de diferentes servicios online - RedesZone, fecha de acceso: noviembre 16, 2025, <https://www.redeszone.net/tutoriales/configuracion-puertos/puertos-tcp-udp/>
7. TCP congestion control - Wikipedia, fecha de acceso: noviembre 16, 2025, https://en.wikipedia.org/wiki/TCP_congestion_control
8. UDP - User Datagram Protocol - Networkgeeks, fecha de acceso: noviembre 16, 2025, <https://netwgeeks.com/udp-user-datagram-protocol/>
9. Difference between SCTP and TCP - GeeksforGeeks, fecha de acceso: noviembre 16, 2025, <https://www.geeksforgeeks.org/computer-networks/difference-between-sctp-and-tcp/>
10. QUIC - Wikipedia, fecha de acceso: noviembre 16, 2025, <https://en.wikipedia.org/wiki/QUIC>
11. draft-joseph-quic-comparison-quic-sctp-00 - IETF Datatracker, fecha de acceso: noviembre 16, 2025, <https://datatracker.ietf.org/doc/html/draft-joseph-quic-comparison-quic-sctp-00>
12. QUIC- Will it Replace TCP/IP? - SNIA.org, fecha de acceso: noviembre 16, 2025, <https://www.snia.org/sites/default/files/ESF/QUIC-Will-It-Replace-TCP-IP-Final.pdf>
13. Visión general de la interfaz de biblioteca de servicios de transporte - IBM, fecha de acceso: noviembre 16, 2025, <https://www.ibm.com/docs/es/aix/7.3.0?topic=streams-transport-service-library-interface-overview>
14. Capa de Transporte, fecha de acceso: noviembre 16, 2025, <https://www.cartagena99.com/recursos/alumnos/apuntes/Tema06.pdf>
15. Encapsulado de datos y la pila de protocolo TCP/IP (Guía de administración del sistema, fecha de acceso: noviembre 16, 2025, <https://docs.oracle.com/cd/E19957-01/820-2981/6nei0r0s7/index.html>
16. Capa de Internet: preparación de los paquetes para la entrega, fecha de acceso: noviembre 16, 2025, <https://docs.oracle.com/cd/E19957-01/820-2981/ipov-38/index.html>
17. 7.1.2.10 Segmentación TCP y UDP - Institut Sa Palomera, fecha de acceso: noviembre 16, 2025, <https://www.sapalomera.cat/moodlecf/RS/1/course/module7/7.1.2.10/7.1.2.10.html>

18. HEADER TCP - UDP (Encabezados) y Puertos - YouTube, fecha de acceso: noviembre 16, 2025, https://www.youtube.com/watch?v=41_xKy773OY
19. TCP 3-Way Handshake Process - GeeksforGeeks, fecha de acceso: noviembre 16, 2025, <https://www.geeksforgeeks.org/computer-networks/tcp-3-way-handshake-process/>
20. TCP 3-Way Handshake Process. Transmission Control Protocol | by Kusal Kaluarachchi, fecha de acceso: noviembre 16, 2025, <https://medium.com/@kusal95/tcp-3-way-handshake-process-1fd9a056a2f4>
21. Description of Windows TCP features - Windows Server - Microsoft Learn, fecha de acceso: noviembre 16, 2025, <https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/description-tcp-features>
22. IPv4 vs IPv6 - Understanding the differences | NetworkAcademy.IO, fecha de acceso: noviembre 16, 2025, <https://www.networkacademy.io/ccna/ipv6/ipv4-vs-ipv6>
23. Difference Between IPv4 and IPv6 - GeeksforGeeks, fecha de acceso: noviembre 16, 2025, <https://www.geeksforgeeks.org/computer-networks/differences-between-ipv4-and-ipv6/>
24. FW: ¿Por qué querías mandar un TCP RST cuando algo está bloqueado? - Reddit, fecha de acceso: noviembre 16, 2025, https://www.reddit.com/r/networking/comments/5jr1vw/fw_why_would_you_want_to_send_a_tcp_rst_when/?tl=es-419
25. TCP Resets (TCP RST): Everything You Need to Know About TCP Connections | LogicMonitor, fecha de acceso: noviembre 16, 2025, <https://www.logicmonitor.com/blog/tracking-down-failed-tcp-connections-and-rst-packets>
26. ¿Cómo comprobar si un puerto UDP está realmente abierto? : r/linuxadmin - Reddit, fecha de acceso: noviembre 16, 2025, https://www.reddit.com/r/linuxadmin/comments/17ayyam/how_to_check_if_a_udp_port_is_actually_open/?tl=es-419
27. icmpush - Generador de paquetes ICMP - Ubuntu Manpage, fecha de acceso: noviembre 16, 2025, <https://manpages.ubuntu.com/manpages/focal/es/man8/icmpush.8.html>
28. ¿Qué es el Escaneo de Puertos y Para qué Sirve? Lo Explicamos con Palabras Sencillas, fecha de acceso: noviembre 16, 2025, <https://cybermentor.net/escanear-puertos-guia-completa/>
29. Números de puerto - IBM, fecha de acceso: noviembre 16, 2025, <https://www.ibm.com/docs/es/cics-ts/6.x?topic=concepts-port-numbers>
30. fecha de acceso: noviembre 16, 2025, https://es.wikipedia.org/wiki/Puerto_ef%C3%ADmero#:~:text=Un%20puerto%20ef%C3%ADmero%20es%20usado,servidor%20a%20un%20puerto%20bien
31. Ephemeral port, fecha de acceso: noviembre 16, 2025, https://en.wikipedia.org/wiki/Ephemeral_port

32. TCP 3-Way Handshake | Computer Networks - workat.tech, fecha de acceso: noviembre 16, 2025, <https://workat.tech/core-cs/tutorial/tcp-three-way-handshake-in-computer-networks-yoo7331910lh>
33. TCP handshake - Glossary - MDN Web Docs, fecha de acceso: noviembre 16, 2025, https://developer.mozilla.org/en-US/docs/Glossary/TCP_handshake
34. 7.2.2.2 Confiabilidad de TCP: reconocimiento y tamaño de la ventana, fecha de acceso: noviembre 16, 2025, <https://www.sapalomera.cat/moodlecf/RS/1/course/module7/7.2.2.2/7.2.2.2.html>
35. SYN, SYN-ACK, ACK seguido de FIN-ACK : r/networking - Reddit, fecha de acceso: noviembre 16, 2025, https://www.reddit.com/r/networking/comments/12hgd67/syn_synack_ack_followed_by_finack/?tl=es-419
36. TCP - why tcp choose lower MSS ? - Cisco Learning Network, fecha de acceso: noviembre 16, 2025, <https://learningnetwork.cisco.com/s/question/0D53i00000SngXLCaZ/tcp-why-tcp-choose-lower-mss->
37. Parámetros ajustables TCP - Manual de referencia de parámetros ajustables de Oracle Solaris 11.1, fecha de acceso: noviembre 16, 2025, https://docs.oracle.com/cd/E37929_01/html/E36673/chapter4-31.html
38. TCP Congestion Control - GeeksforGeeks, fecha de acceso: noviembre 16, 2025, <https://www.geeksforgeeks.org/computer-networks/tcp-congestion-control/>
39. ¿Cómo utiliza TCP los números de secuencia en la gestión de segmentos de datos y acuses de recibo? - Academia EITCA, fecha de acceso: noviembre 16, 2025, <https://es.eitca.org/la-seguridad-cibern%C3%A9tica/eitc-es-cnffundamentos-de-redes-inform%C3%A1ticas/protocolos-de-internet/c%C3%B3mo-tcp-manage-los-errores-y-usa-Windows/examen-revisa-c%C3%B3mo-TCP-manage-errores-y-usa-Windows/%C2%BFC%C3%B3mo-utiliza-TCP-los-n%C3%BAmeros-de-secuencia-en-la-gesti%C3%B3n-de-segmentos-de-datos-y-reconocimientos%3F/>
40. netstat - What's the difference between port status "LISTENING ...", fecha de acceso: noviembre 16, 2025, <https://askubuntu.com/questions/538443/whats-the-difference-between-port-status-listening-time-wait-close-wait>
41. What are CLOSE_WAIT and TIME_WAIT states? - Super User, fecha de acceso: noviembre 16, 2025, <https://superuser.com/questions/173535/what-are-close-wait-and-time-wait-states>
42. Socket programming, what about "CLOSE_WAIT", "FIN_WAIT_2" and "LISTENING"?, fecha de acceso: noviembre 16, 2025, <https://stackoverflow.com/questions/2782081/socket-programming-what-about-close-wait-fin-wait-2-and-listening>
43. TRABAJO FIN DE GRADO - Biblos-e Archive, fecha de acceso: noviembre 16, 2025, https://repositorio.uam.es/bitstream/handle/10486/688414/dompablo_tobar_javie

[r_tfg.pdf?s](#)

44. A ferramenta PortQry 2.0 - TechNet | Microsoft Learn, fecha de acceso: noviembre 16, 2025, <https://learn.microsoft.com/pt-br/security-updates/security/20113596>
45. What do the UDP entries in my netstat output stand for?, fecha de acceso: noviembre 16, 2025, <https://security.stackexchange.com/questions/13724/what-do-the-udp-entries-in-my-netstat-output-stand-for>
46. netstat command shows SYN_RECV - Stack Overflow, fecha de acceso: noviembre 16, 2025, <https://stackoverflow.com/questions/11973022/netstat-command-shows-syn-rec-v>
47. All connections from this network get stuck in SYN_RECV state, connections from my home or phone properly get ESTABLISHED - Server Fault, fecha de acceso: noviembre 16, 2025, <https://serverfault.com/questions/273807/all-connections-from-this-network-get-stuck-in-syn-recv-state-connections-from>
48. 7.5. TCP Analysis - Wireshark, fecha de acceso: noviembre 16, 2025, https://www.wireshark.org/docs/wsug_html_chunked/ChAdvTCPAnalysis.html
49. Introduction to Network Trace Analysis 3: TCP Performance ..., fecha de acceso: noviembre 16, 2025, <https://techcommunity.microsoft.com/blog/coreinfrastructureandsecurityblog/introduction-to-network-trace-analysis-3-tcp-performance/3737115>
50. How TCP really works: MTU vs MSS - YouTube, fecha de acceso: noviembre 16, 2025, <https://www.youtube.com/watch?v=J-gnDC6B5eE>
51. Wireshark 101: TCP Flow Control, HakTip 135 - YouTube, fecha de acceso: noviembre 16, 2025, <https://www.youtube.com/watch?v=McDNzBvRPHA>
52. Why would "TCP Full Window" happen? - Wireshark Q&A, fecha de acceso: noviembre 16, 2025, <https://osqa-ask.wireshark.org/questions/24501/why-would-tcp-full-window-happen/>
53. The transmission window is now completely full - Wireshark Q&A, fecha de acceso: noviembre 16, 2025, <https://osqa-ask.wireshark.org/questions/40461/the-transmission-window-is-now-completely-full/>
54. TCP Congestion Control Explained 2.0 | Wireshark | Algorithm - YouTube, fecha de acceso: noviembre 16, 2025, <https://www.youtube.com/watch?v=luF5MWuw1DY>
55. TCP Congestion Control // Hands-On Deep Dive TCP Analysis with Wireshark - YouTube, fecha de acceso: noviembre 16, 2025, <https://www.youtube.com/watch?v=IRXP1vJ6-vM>
56. Go-Back-N ARQ - Wikipedia, fecha de acceso: noviembre 16, 2025, https://en.wikipedia.org/wiki/Go-Back-N_ARQ
57. Simulación de los mecanismos de control de congestión en TCP/IP - Universidad Politécnica de Cartagena, fecha de acceso: noviembre 16, 2025,

- <https://www.upct.es/~orientap/TCP.pdf>
58. A Tour Around TCP -- The RTT Variance Estimation, fecha de acceso: noviembre 16, 2025, https://tnlandforms.us/tcptour/javis/tcp_rttvar.html
 59. 4.2.3.1 Retransmission Timeout Calculation - freesoft.org, fecha de acceso: noviembre 16, 2025, <https://www.freesoft.org/CIE/RFC/1122/109.htm>
 60. www.rfc-editor.org, fecha de acceso: noviembre 16, 2025, <https://www.rfc-editor.org/rfc/rfc6298.txt>
 61. Slow Start vs Congestion Avoidance in TCP - YouTube, fecha de acceso: noviembre 16, 2025, <https://www.youtube.com/watch?v=r9kbjAN2788>
 62. TCP Congestion Control, fecha de acceso: noviembre 16, 2025, <https://www.cs.cmu.edu/~prs/15-441-F14/lectures/rec05-congestion.pdf>
 63. TCP Tahoe and TCP Reno - GeeksforGeeks, fecha de acceso: noviembre 16, 2025, <https://www.geeksforgeeks.org/computer-networks/tcp-tahoe-and-tcp-reno/>
 64. TCP congestion control - Wikipedia, fecha de acceso: noviembre 16, 2025, https://en.wikipedia.org/wiki/TCP_congestion_control#TCP_Tahoe_and_Reno
 65. network programming - TCP congestion control - Fast Recovery in graph - Stack Overflow, fecha de acceso: noviembre 16, 2025, <https://stackoverflow.com/questions/30818925/tcp-congestion-control-fast-recovery-in-graph>
 66. Traducción de direcciones de red - Wikipedia, la enciclopedia libre, fecha de acceso: noviembre 16, 2025, https://es.wikipedia.org/wiki/Traducci%C3%B3n_de_direcciones_de_red
 67. What is the difference between a Source NAT, Destination NAT and Masquerading?, fecha de acceso: noviembre 16, 2025, <https://serverfault.com/questions/119365/what-is-the-difference-between-a-source-nat-destination-nat-and-masquerading>
 68. masquerading vs NAT - Beginner Basics - MikroTik community forum, fecha de acceso: noviembre 16, 2025, <https://forum.mikrotik.com/t/masquerading-vs-nat/40530>
 69. About NAT and Azure VPN Gateway - Microsoft Learn, fecha de acceso: noviembre 16, 2025, <https://learn.microsoft.com/en-us/azure/vpn-gateway/nat-overview>
 70. netstat-nat(1) - Arch manual pages, fecha de acceso: noviembre 16, 2025, <https://man.archlinux.org/man/extra/netstat-nat/netstat-nat.1.en>
 71. Dominando el Comando Linux Netstat: De lo Básico al Monitoreo Avanzado de Redes, fecha de acceso: noviembre 16, 2025, <https://go.lightnode.com/es/tech/linux-netstat-command>
 72. How to Simulate a TCP/UDP Client Using Netcat - Ubidots, fecha de acceso: noviembre 16, 2025, <https://ubidots.com/blog/how-to-simulate-a-tcpudp-client-using-netcat/>
 73. Listen to UDP data on local port with netcat - Server Fault, fecha de acceso: noviembre 16, 2025, <https://serverfault.com/questions/207683/listen-to-udp-data-on-local-port-with-netcat>

74. Using netcat to send a UDP packet without binding - GeeksforGeeks, fecha de acceso: noviembre 16, 2025,
<https://www.geeksforgeeks.org/linux-unix/using-netcat-to-send-a-udp-packet-without-binding/>
75. Why UDP does not show LISTENING in the state column in netstat? - Super User, fecha de acceso: noviembre 16, 2025,
<https://superuser.com/questions/1096504/why-udp-does-not-show-listening-in-the-state-column-in-netstat>
76. netstat | Microsoft Learn, fecha de acceso: noviembre 16, 2025,
<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/netstat>
77. netstat -na : udp and state established? - Stack Overflow, fecha de acceso: noviembre 16, 2025,
<https://stackoverflow.com/questions/10841678/netstat-na-udp-and-state-established>
78. inetd Daemon - IBM, fecha de acceso: noviembre 16, 2025,
https://www.ibm.com/docs/ssw_aix_71/i_commands/inetd.html
79. Security considerations for INETD server on IBM i, fecha de acceso: noviembre 16, 2025,
<https://www.ibm.com/docs/en/i/7.4.0?topic=security-considerations-inetd-server>
80. inetutils-inetd — Debian testing, fecha de acceso: noviembre 16, 2025,
<https://manpages.debian.org/testing/inetutils-inetd/inetutils-inetd.8.en.html>
81. UDP socket programming using python - Codemia, fecha de acceso: noviembre 16, 2025,
https://codemia.io/knowledge-hub/path/udp_socket_programming_using_python
82. UDP Communication - Python Wiki, fecha de acceso: noviembre 16, 2025,
<https://wiki.python.org/moin/UdpCommunication>
83. COMO FUNCIONA TRACEROUTE. Pregunta de Entrevista [Nuevo CCNA][Tracert] . Explicación completa (2020) - YouTube, fecha de acceso: noviembre 16, 2025,
<https://www.youtube.com/watch?v=uUIPiBZkmlk>
84. ¿Qué es Traceroute? - Dotcom-Monitor, fecha de acceso: noviembre 16, 2025,
<https://www.dotcom-monitor.com/es/aprende-con-dotcom-monitor/glosario/que-es-traceroute/>
85. socket — Low-level networking interface — Python 3.14.0 documentation, fecha de acceso: noviembre 16, 2025, <https://docs.python.org/3/library/socket.html>
86. Socket Programming HOWTO — Python 3.14.0 documentation, fecha de acceso: noviembre 16, 2025, <https://docs.python.org/3/howto/sockets.html>
87. Socket Programming in Python (Guide), fecha de acceso: noviembre 16, 2025,
<https://realpython.com/python-sockets/>
88. python - How to close a socket left open by a killed program? - Stack Overflow, fecha de acceso: noviembre 16, 2025,
<https://stackoverflow.com/questions/5875177/how-to-close-a-socket-left-open-by-a-killed-program>
89. Chapter 32. Network Servers | FreeBSD Documentation Portal, fecha de acceso: noviembre 16, 2025,

- <https://docs.freebsd.org/en/books/handbook/network-servers/>
90. inetd - QNX, fecha de acceso: noviembre 16, 2025,
https://www.qnx.com/developers/docs/6.5.0SP1.update/com.qnx.doc.neutrino_utilities/i/inetd.html
 91. How to use Telnet and Netcat commands to test Port connectivity - eukhost, fecha de acceso: noviembre 16, 2025,
<https://www.eukhost.com/kb/how-to-use-telnet-and-netcat-commands-to-test-port-connectivity/>
 92. Basic troubleshooting with telnet and netcat - Red Hat, fecha de acceso: noviembre 16, 2025,
<https://www.redhat.com/en/blog/telnet-netcat-troubleshooting>
 93. Use Netcat to Establish and Test TCP and UDP Connections: A ..., fecha de acceso: noviembre 16, 2025,
<https://www.digitalocean.com/community/tutorials/how-to-use-netcat-to-establish-and-test-tcp-and-udp-connections>
 94. Tiempo de espera de inactividad y restablecimiento de TCP de Load Balancer en Azure - Microsoft Learn, fecha de acceso: noviembre 16, 2025,
<https://learn.microsoft.com/es-es/azure/load-balancer/load-balancer-tcp-reset>
 95. TIME_WAIT state | Back To The Basics, fecha de acceso: noviembre 16, 2025,
<https://totozhang.github.io/2016-01-31-tcp-timewait-status/>
 96. Setting TCP timed wait delay (TcpTimedWaitDelay) - IBM, fecha de acceso: noviembre 16, 2025,
<https://www.ibm.com/docs/en/records-manager/8.5.0?topic=performance-setting-tcp-timed-wait-delay-tcptimedwaitdelay>
 97. Why TIME_WAIT state need to be 2MSL long? - Stack Overflow, fecha de acceso: noviembre 16, 2025,
<https://stackoverflow.com/questions/25338862/why-time-wait-state-need-to-be-2msl-long>